

Licenciatura em Engenharia Informática

UC | Projeto de Desenvolvimento Móvel

Docente | Pedro Rosa

Proposta inicial do projeto: Gestor de Finanças pessoais para Estudantes “Studycash”

Autores:

Samuel da Graça Soares Ferreira- 20220755

Marlinda da Conceição Prade Congo- 20241718

Lueji Rafael Silvestre Covilhã- 20241725

Constantino de Almeida Chipopa- 20241231

Gilma Hortência de Sousa Mulanda- 20241087

Lisboa,

09 de Novembro 2025

Palavras chaves

Estudantes

Controle de gastos

Planeamento financeiro

Economia

Gestão financeira

Introdução

O projeto baseia-se no desenvolvimento de uma aplicação móvel intitulada Studycash, que ajuda estudantes a gerir as suas finanças pessoais de forma simples e prática. Incentivando decisões financeiras conscientes, o hábito de economizar e a construção de uma base financeira estável para o futuro.

O problema que este projeto procura resolver é a falta de ferramentas simples e adaptadas à rotina dos estudantes para o controlo do dinheiro. Ao disponibilizar uma solução digital fácil de usar, espera-se promover hábitos financeiros mais conscientes, melhorar o planeamento dos recursos e incentivar uma relação mais equilibrada e responsável com o dinheiro no dia a dia académico.

Objetivo

O principal objetivo deste projeto é desenvolver um aplicativo móvel para gestão de finanças pessoais direcionado a estudantes, oferecendo uma ferramenta prática e acessível que auxilie no planeamento e controlo dos recursos financeiros.

Para alcançar esse objetivo geral, definem-se os seguintes objetivos específicos:

- Permitir o registo de receitas e despesas de forma simples e organizada;
- Facilitar a criação e acompanhamento de orçamentos mensais, ajudando o utilizador a controlar melhor os seus gastos;
- Proporcionar a definição de metas de poupança, incentivando a construção de hábitos financeiros saudáveis;
- Disponibilizar relatórios e gráficos que apresentem a evolução financeira ao longo do tempo;
- Promover a educação financeira entre estudantes, contribuindo para uma gestão mais consciente e responsável do dinheiro.

Motivação

A motivação para a escolha deste tema surge da realidade financeira enfrentada por muitos estudantes, que frequentemente têm dificuldade em gerir o seu dinheiro de forma adequada. A falta de planeamento e de ferramentas adequadas pode levar a desequilíbrios financeiros, endividamento e à incapacidade de alcançar objetivos pessoais ou académicos. Com o aumento do uso de smartphones e da tecnologia no dia a dia, um aplicativo móvel representa uma solução acessível e eficaz para ajudar os estudantes a acompanhar e organizar as suas finanças. Além disso, o projeto pretende incentivar a educação financeira desde cedo, proporcionando aos jovens as competências necessárias para tomar decisões mais conscientes e responsáveis sobre o uso do dinheiro.

Público-alvo

A aplicação é destinada a estudantes do ensino secundário e superior que procuram gerir as suas finanças pessoais de forma organizada e eficiente. Além dos estudantes, a aplicação também pode ser útil para jovens em início de vida profissional, contribuindo para o desenvolvimento de hábitos financeiros responsáveis e para uma melhor planeação económica pessoal.

Aplicações existentes

Temos como exemplo as aplicações abaixo que apresentam funcionalidades e objetivos alinhados com os propostos no Studycash.

- SpendeeSpendee
- MoneyManeger
- SimpleMoney
- You need a budget (YNAB)

Guiões

Menu principal

Passos:

- Abre o aplicativo → é direcionado para a Página Inicial.
- Vê o saldo atual e o resumo das receitas e despesas.
- Analisa as transações recentes (ex.: “Almoço RU”, “Mesada”).
- Clica em “Ver todas” para abrir o histórico completo.
- Observa se está a gastar mais do que recebe e planeia ajustes futuros.

Histórico de Transações

Passos:

- No menu inferior, seleciona “Transações”.
- Vê os totais de Receitas e Despesas destacados no topo.
- Usa o campo “Buscar transações...” para encontrar algo específico.
- Filtra por “Todas”, “Receitas” ou “Despesas”.
- Analisa cada item com data, categoria e valor (ex.: “Material Escolar -45,50 €”).

Adicionar Nova Transação

Passos:

- Toca no botão “+” do menu inferior.
 - Escolhe o tipo de transação:
 - Despesa (ex.: almoço, transporte)
 - Receita (ex.: mesada, salário)
1. Preenche os campos:
 - a. Título (ex.: “Almoço no RU”)
 - b. Valor (ex.: “12,50 €”)

- c. Categoria (ex.: “Alimentação”)
 - d. Data
2. Clica em “Adicionar Despesa” ou “Adicionar Receita”.
3. A transação aparece automaticamente na lista e atualiza o saldo.

Orçamento

Passos:

- No menu inferior, seleciona “Orçamento”.
- Observa o Gasto Total e o Orçamento Planeado.
- Acompanha a barra de progresso para ver o percentual utilizado.
- Analisa o gráfico de Distribuição de Gastos por categoria (Alimentação, Educação, Transporte, etc.).
- Verifica na seção “Por Categoria” quais áreas estão a consumir mais do orçamento.
- Faz ajustes nos gastos para manter-se dentro dos limites mensais

Metas

Passos:

- No menu inferior, toca em “Metas”.
- Preenche o formulário com:
 - Título (ex.: “Viagem”)
 - Valor da meta (ex.: “220 €”)
 - Categoria e Data.
- Clica em “Adicionar Meta”.
- Acompanha o progresso (ex.: “70 € de 220 € – 70%”).
- Usa essa visualização para motivar-se a economizar e atingir o objetivo

Perfil

Passos:

1. Clica no ícone “Perfil” no canto superior direito.

2. Visualiza seus dados:
 - a. Nome, e-mail e área de estudo.
3. Consulta estatísticas gerais:
 - a. Total de transações realizadas e valor economizado.
4. Ativa ou desativa notificações financeiras.
5. Entra em Metas Financeiras para editar objetivos.
6. Clica em “Sair da Conta” quando quiser encerrar a sessão.

Descrição genérica da solução a implementar

A solução proposta consiste no desenvolvimento de um aplicativo móvel multiplataforma voltado para a gestão de finanças pessoais de estudantes. O sistema permitirá que os utilizadores registem receitas e despesas, definam orçamentos mensais, estabeleçam metas de poupança e consultem relatórios e gráficos financeiros de forma simples e acessível. A aplicação será desenvolvida com base em programação orientada a objetos e utilizará uma base de dados relacional para armazenar e organizar as informações financeiras com segurança. A interface será intuitiva e responsiva, facilitando a navegação e o uso diário.

Enquadramento nas unidades curriculares

O desenvolvimento deste projeto envolve cinco unidades curriculares, cada uma desempenhando papéis específicos e contribuindo de diferentes formas, das quais podemos destacar:

- Matemática Discreta: Esta UC contribui diretamente para o desenvolvimento do projeto ao fornecer as bases lógicas e estruturais necessárias para o planeamento e funcionamento do aplicativo. Utilizando conceitos como lógica e álgebra booleana.
- Programação de dispositivos móveis: Representa a base técnica do projeto, pois será responsável pela criação da interface do aplicativo utilizando as tecnologias Java/Kotlin e jetpack compose.
- Programação orientada a objetos: Contribuirá para a implementação de servidores que asseguram um fluxo de informações mais eficiente e estruturado, através da utilização de frameworks como o Spring boot, que facilitam a construção, integração e manutenção dos serviços back-end do aplicativo.
- Base de dados: Esta UC será responsável pela criação e gestão da base de dados que armazenará todas as informações dos utilizadores, permitindo o seu acesso, atualização e manipulação conforme as necessidades de funcionamento da aplicação. Para isso, será utilizada a tecnologia SQL, garantindo uma estrutura de dados eficiente, segura e organizada.

- **Competências Comunicacionais:** Esta UC será responsável por garantir a aplicação de métodos e técnicas que possibilitarão uma apresentação mais eficaz do projeto, seguindo as melhores práticas de comunicação e transmissão de informações.

Requisitos Técnicos para o desenvolvimento do projeto

1. Plataforma e Ambiente de Desenvolvimento

- Aplicação Android desenvolvida em Kotlin com Jetpack Compose.
- Backend em Java com Spring Boot, arquitetura MVC e API REST.
- Ferramentas: Android Studio Koala 2024.1.2, IntelliJ/Eclipse, Maven/Gradle.

2. Linguagens de programação

- Java (backend), Kotlin (frontend) e SQL (base de dados).

3. Banco de Dados

- Relacional: PostgreSQL ou MySQL.
- Tabelas: usuários, receitas, despesas, orçamentos e metas.
- Implementação via Spring Data JPA e CRUD completo.

4. Interface (UI/UX)

- Desenvolvida em Jetpack Compose, baseada no Material Design 3.
- Mockups obrigatórios no Figma.
- Interface responsiva, intuitiva e com gráficos interativos.

5. Segurança

- Login com e-mail e senha.
- Criptografia (BCrypt) e HTTPS.
- Validação de dados no cliente e servidor.

6. Gestão e Versionamento

- **GitHub:** código e documentação.
- **ClickUp:** gestão de tarefas.
- **Discord:** comunicação da equipa.

- **Figma:** protótipos.

7. Documentação

- Diagrama de classes, modelo ER e documentação REST (padrão Bocoup).
- Manual técnico e do utilizador incluídos no repositório GitHub.

Arquitetura de Solução

Temos três camadas para a arquitetura de solução:

1. Camada de apresentação (Frontend): responsável pela interface do usuário.
2. Camada de lógica de Negócio (Backend): servidor e lógica da aplicação.
3. Base de dados: responsável pelo armazenamento de todas as informações do sistema.

Fronted

O frontend da aplicação será desenvolvido no Android Studio, utilizando a linguagem Kotlin e o framework Jetpack Compose como suporte. A interface do utilizador permitirá gerir orçamentos e despesas, e realizar análises financeiras com base nos gráficos gerados pela aplicação.

Backend

O backend da aplicação, responsável pelas APIs e pelo servidor, será desenvolvido em Java utilizando o framework Spring Boot. Além de gerir as APIs e o servidor, o Spring Boot será utilizado para estabelecer as conexões entre o frontend e o servidor, e entre o servidor e a base de dados, através da implementação de APIs RESTful. O backend terá como principal função processar as requisições recebidas do frontend e gerir as operações do sistema, sendo implementado com uma API simples e eficiente.

Base de dados

A base de dados será desenvolvida com MySQL, utilizando a linguagem SQL, e contará com diversos mecanismos de proteção de dados integrados através do Spring Boot. Ela será responsável por armazenar todas as informações relacionadas a orçamentos, despesas e utilizadores, garantindo a segurança, integridade e disponibilidade dos dados.

Tecnologias a utilizar

O desenvolvimento do projeto conta com as seguintes tecnologias:

Frameworks: Spring Boot

Website: Figma

Linguagens: Java, Kotlin e SQL

Aplicações: MYSQL Workbench, Github, Discord, Clickup, Visual Studio Code, Android Studio.

Planeamento e Calendarização

Tarefas	Setembro				Outubro				Novembro				Dezembro				Responsáveis
	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	Semana 1	Semana 2	Semana 3	Semana 4	
Planeamento e Pesquisa																	Samuel, Marlinda
Design																	Samuel, Marlinda, Constantino, Lueji, Gilma
Desenvolvimento																	Samuel, Marlinda, Constantino, Lueji, Gilma
Testes																	Samuel, Marlinda, Constantino, Lueji, Gilma
Documentação																	Samuel, Marlinda, Constantino, Lueji, Gilma
Apresentação e Entrega																	Samuel, Marlinda, Constantino, Lueji, Gilma

Personas

O Gustavo, o Estudante Financeiramente Organizado

Idade: 21 anos

Curso: Engenharia de Software

Nível de conhecimento financeiro: Avançado

Objetivos

- Controlar receitas e despesas mensais com precisão.
- Acompanhar metas de poupança (ex.: economizar para comprar um computador novo).
- Analisar gráficos e relatórios financeiros para otimizar gastos.

Comportamento e Frustrações

- Usa planilhas ou apps bancários, mas sente falta de uma ferramenta central e visual.
- Às vezes perde tempo ajustando categorias e valores manualmente.
- Quer evitar gastos repetitivos e compras impulsivas.

Motivações para usar o StudyCash

- Deseja um app intuitivo e analítico para gerenciar seu orçamento estudantil.
- Quer relatórios automáticos e alertas de limite de orçamento.
- Valoriza um design minimalista e funcional, que facilite a comparação entre meses.

Luzia, a Estudante Financeiramente Iniciante

Idade: 22 anos

Curso: comunicação Social

Nível de conhecimento financeiro: Básico

Objetivos

- Aprender a gerir o próprio dinheiro de forma simples e prática.
- Evitar gastar mais do que ganha e criar hábitos de poupança.
- Ter uma visão clara de onde o dinheiro está sendo gasto.

Comportamento e Frustrações

- Costuma gastar por impulso e não acompanha seus gastos.
- Fica surpresa ao final do mês por não saber para onde o dinheiro foi.
- Acha os aplicativos financeiros difíceis e confusos, cheios de termos técnicos.

Motivações para usar o StudyCash

- Busca um app educativo e simples, com sistema de categorização automática de despesas para compreender melhor onde gastar mais.
- Aprecia visualizar gráficos interativos e relatórios mensais que mostram a evolução dos gastos.
- Valoriza receber notificações de limite de orçamento e definir metas financeiras que ajudam a manter o controlo.

Casos de Utilização

Transações

Ator principal: Utilizador

Objetivo: Permitir ao utilizador visualizar, pesquisar e filtrar todas as transações financeiras registadas no sistema.

Descrição: Este caso de utilização permite ao utilizador consultar o histórico completo de transações (receitas e despesas), visualizar totais, aplicar filtros e analisar a evolução das finanças pessoais.

Pré- Condições

- O utilizador encontra-se autenticado no sistema.
- Existem transações previamente registadas.

Fluxo Principal

1. O Utilizador acede à secção **Transações**.
2. O sistema apresenta o resumo financeiro com:
 - Total de receitas
 - Total de despesas
3. O sistema apresenta a lista de transações ordenada por data.
4. O utilizador visualiza os detalhes básicos de cada transação (descrição, tipo, valor e data).

Fluxos Alternativos

- **Filtrar transações**
 1. O utilizador seleciona um filtro (**Todas**, **Receitas** ou **Despesa**).
 2. O Sistema atualiza a lista conforme o filtro selecionado.
- **Pesquisar transações**
 1. O utilizador insere um termo no campo **Pesquisar transações**.

2. O sistema apresenta apenas as transações correspondentes à pesquisa.

Pós- Condições

- As transações são apresentadas corretamente de acordo com os filtros aplicados.
- O utilizador obtém uma visão clara do seu histórico financeiro.

Regras de Negócio

- As receitas são apresentadas com valores positivos.
- As despesas são apresentadas com valores negativos.
- Os totais de receitas e despesas correspondem à soma das transações registadas.
- A lista deve ser atualizada em tempo real após novos registos.

Nova Receita

Ator principal: Utilizador.

Objetivo: permitir ao utilizador registar uma nova receita no sistema , de forma a atualizar o saldo e controlo financeiro.

Descrição: este caso de utilização permite ao utilizador registar um ganho financeiro, indicando valor, descrição, data e localização, garantindo o correto acompanhamento das receitas.

Pré- Condições

- O utilizador encontra-se autenticado no sistema.
- Existem categorias registadas.
- Existe pelo menos uma carteira associada ao utilizador.

Fluxo Principal

1. O utilizador seleciona a opção **Adicionar Transação**.
2. O utilizador escolhe o tipo **Receita**.
3. O sistema apresenta o formulário **Nova Receita**.
4. O utilizador introduz o **valor da receita**.
5. O utilizador preenche a **descrição**.
6. O utilizador seleciona uma **categoria**.
7. O utilizador define a **data da receita**.
8. O sistema captura a **localização** (opcional).
9. O utilizador seleciona **Adicionar Receita**.
10. O sistema valida os dados.
11. O sistema regista a receita e atualiza o saldo.

Fluxos Alternativos

- **Dados inválidos:**
 - Se o valor for nulo ou negativo, o sistema informa o utilizador e solicita correção.
- **Localização indisponível:**
 - Se a localização não puder ser capturada, a receita é registada sem esse dado.

Pós- Condições

- A receita fica registada no sistema.
- O saldo e os resumos financeiros são atualizados.

Regras de Negócio

- O valor da receita deve ser maior que zero.
- A receita deve estar associada a uma categoria.
- O tipo da transação é definido como **Receita**.

Nova Despesa

Ator principal: utilizador.

Objetivo: permitir ao utilizador registar uma nova despesa, possibilitando o controlo dos gastos e de orçamento.

Descrição: este caso de utilização permite ao utilizador registar um gasto financeiro, garantindo o correto cálculo dos gastos, do orçamento utilizado e do saldo disponível.

Pré- Condições

- O utilizador encontra-se autenticado no sistema.
- Existem categorias registadas.
- Existe orçamento definido (opcional).

Fluxo Principal

1. O utilizador seleciona a opção **Adicionar Transação**.
2. O utilizador escolhe o tipo **Despesa**.
3. O sistema apresenta o formulário **Nova Despesa**.
4. O utilizador introduz o **valor da despesa**.
5. O utilizador preenche a **descrição**.
6. O utilizador seleciona uma **categoria**.
7. O utilizador define a **data da despesa**.
8. O sistema captura a **localização** (opcional).
9. O utilizador seleciona **Adicionar Despesa**.
10. O sistema valida os dados.
11. O sistema regista a despesa e atualiza o orçamento e o saldo.

Fluxos Alternativos

- **Dados inválidos:**
 - Se o valor for nulo ou negativo, o sistema impede o registo.
- **Orçamento excedido:**
 - Se a despesa ultrapassar o orçamento definido, o sistema pode emitir um alerta.

Pós- Condições

- A despesa fica registada no sistema
- O total de gastos e o orçamento utilizado são atualizados.

Regras de Negócio

- O valor da despesa deve ser maior que zero.
- O tipo da transação é definido como **Despesa**.
- As despesas afetam diretamente o orçamento mensal.

Orçamento

Ator principal: utilizador.

Objetivo: definir, atualizar e acompanhar o orçamento mensal, controlando gastos e receitas.

Descrição: este caso de utilização permite ao utilizador definir um limite mensal de gastos, visualizar o valor já utilizado, o montante disponível e analisar receitas e despesas através de indicadores e gráficos.

Pré- Condições

- O utilizador encontra-se autenticado no sistema.
- O utilizador possui transações (receitas e/ou despesas) registadas.

Fluxo Principal

1. O utilizador acede à secção **Orçamento**.
2. O sistema apresenta o mês atual e o resumo financeiro (gastos, receitas e orçamento).
3. O utilizador insere ou altera o **limite mensal de gastos**.
4. O utilizador seleciona a opção **“Atualizar”**.
5. O sistema valida o valor introduzido.
6. O sistema guarda o novo limite de orçamento.
7. O sistema recalcula:
 - Percentagem de orçamento utilizado
 - Valor disponível
8. O sistema apresenta os dados atualizados e os gráficos correspondentes.

Fluxos Alternativos

- **Valor inválido:**
 - Se o valor introduzido for negativo ou nulo, o sistema informa o utilizador e solicita correção.
- **Limite ultrapassado:**
 - Se os gastos excederem o orçamento definido, o sistema pode emitir um alerta visual.

Pós- Condições

- O orçamento mensal fica atualizado.
- O utilizador consegue acompanhar o controlo financeiro do mês.
- Os gráficos e indicadores refletem corretamente os dados registados.

Regras de Negócio

- O limite mensal deve ser um valor positivo.
- O orçamento é associado a um período mensal.
- Os gastos acumulados são calculados com base nas transações do tipo despesa.

Metas

Ator principal: utilizador.

Objetivo: permitir ao utilizador criar uma meta financeira, definindo um objetivo de poupança com valor, categoria e data limite.

Descrição: este caso de utilização permite ao utilizador criar uma nova meta financeira, facilitando o planeamento e o acompanhamento de objetivos pessoais, como poupar para estudos, viagens ou outras necessidades.

Pré- Condições

- O utilizador encontra-se autenticado no sistema.
- Existem categorias previamente registadas.

Fluxo Principal

1. O utilizador acede à secção **Metas**.
2. O sistema apresenta o formulário **Nova Meta**.
3. O utilizador introduz o **nome da meta**.
4. O utilizador define o **valor objetivo**.
5. O utilizador seleciona uma **categoria**.
6. O utilizador escolhe a **data limite**.
7. O utilizador seleciona a opção “**Criar Meta**”.
8. O sistema valida os dados introduzidos.

9. O sistema regista a meta no sistema.
10. O sistema apresenta a meta criada para acompanhamento.

Fluxos Alternativos

- **Valor inválido:**
 - Se algum campo estiver vazio ou o valor objetivo for inválido (≤ 0), o sistema informa o utilizador e solicita correção.
- **Data limite inválida:**
 - Se a data limite for anterior à data atual, o sistema impede a criação da meta.

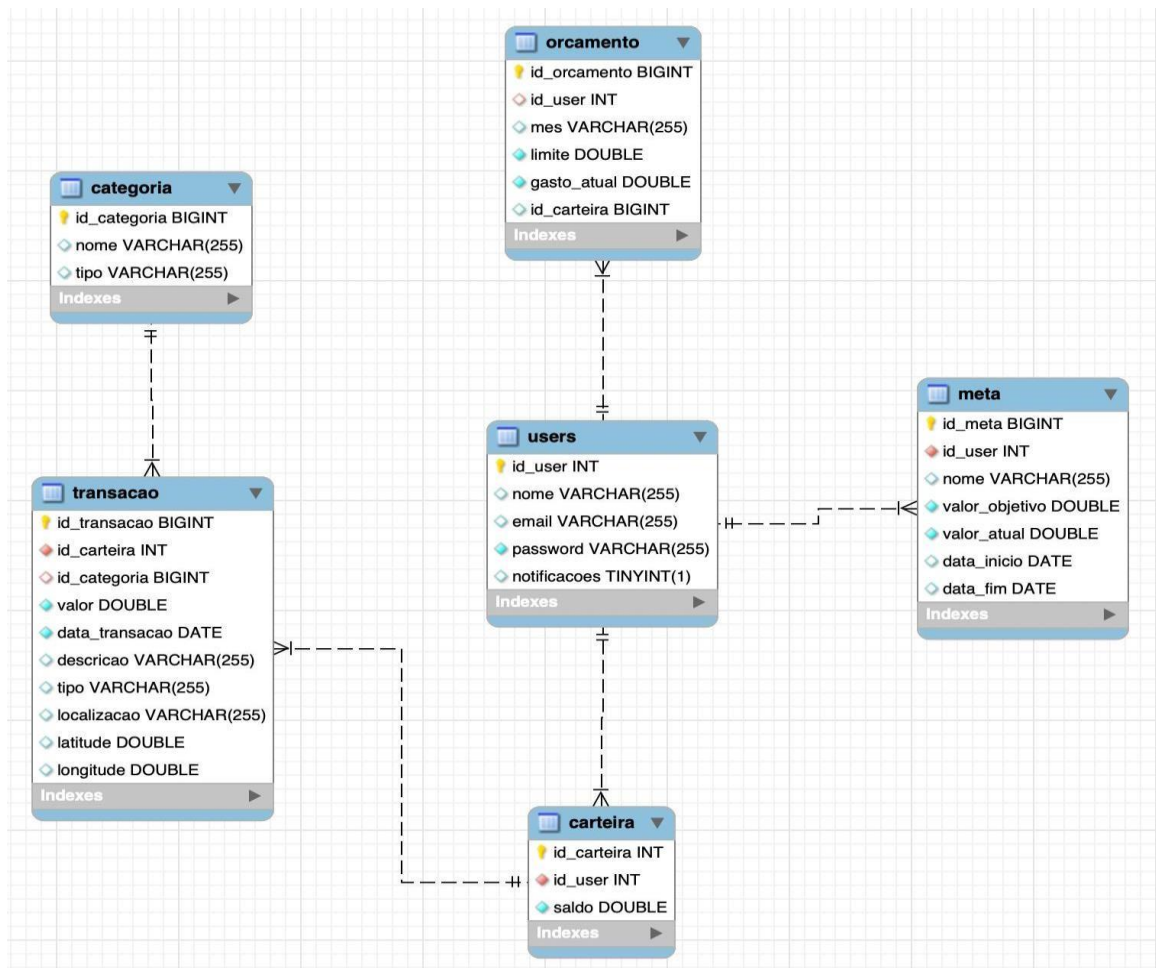
Pós- Condições

- A meta financeira fica registada.
- A meta passa a estar disponível para visualização e acompanhamento.
- O progresso da meta pode ser atualizado com base nas transações registadas.

Regras de Negócio

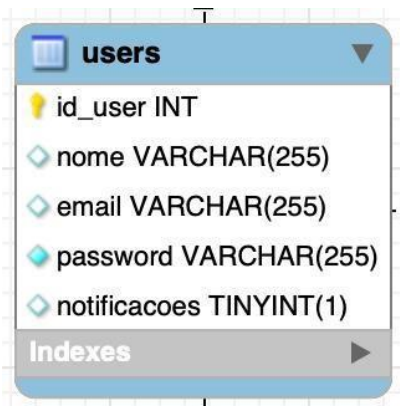
- O valor objetivo deve ser maior que zero.
- A data limite deve ser futura.
- Cada meta está associada a uma categoria.
- O progresso da meta depende das receitas associadas.

Diagrama de classes



Dicionário de dados

Utilizador



A screenshot of a database table definition for the 'users' table. The table has a blue header with the name 'users' and a dropdown arrow. Below the header, the fields are listed: 'id_user' (INT, primary key), 'nome' (VARCHAR(255)), 'email' (VARCHAR(255)), 'password' (VARCHAR(255)), and 'notificacoes' (TINYINT(1)). At the bottom, there is a grey bar labeled 'Indexes' with a right-pointing arrow.

users
id_user INT
nome VARCHAR(255)
email VARCHAR(255)
password VARCHAR(255)
notificacoes TINYINT(1)
Indexes

Metas



A screenshot of a database table definition for the 'meta' table. The table has a blue header with the name 'meta' and a dropdown arrow. Below the header, the fields are listed: 'id_meta' (BIGINT, primary key), 'id_user' (INT), 'nome' (VARCHAR(255)), 'valor_objetivo' (DOUBLE), 'valor_atual' (DOUBLE), 'data_inicio' (DATE), and 'data_fim' (DATE). At the bottom, there is a grey bar labeled 'Indexes' with a right-pointing arrow.

meta
id_meta BIGINT
id_user INT
nome VARCHAR(255)
valor_objetivo DOUBLE
valor_atual DOUBLE
data_inicio DATE
data_fim DATE
Indexes

Categoria



A screenshot of a database table definition for the 'categoria' table. The table has a blue header with the name 'categoria' and a dropdown arrow. Below the header, the fields are listed: 'id_categoria' (BIGINT, primary key), 'nome' (VARCHAR(255)), and 'tipo' (VARCHAR(255)). At the bottom, there is a grey bar labeled 'Indexes' with a right-pointing arrow.

categoria
id_categoria BIGINT
nome VARCHAR(255)
tipo VARCHAR(255)
Indexes

Orçamento

orcamento	
id_orcamento	BIGINT
id_user	INT
mes	VARCHAR(255)
limite	DOUBLE
gasto_atual	DOUBLE
id_carteira	BIGINT
Indexes	

Transações

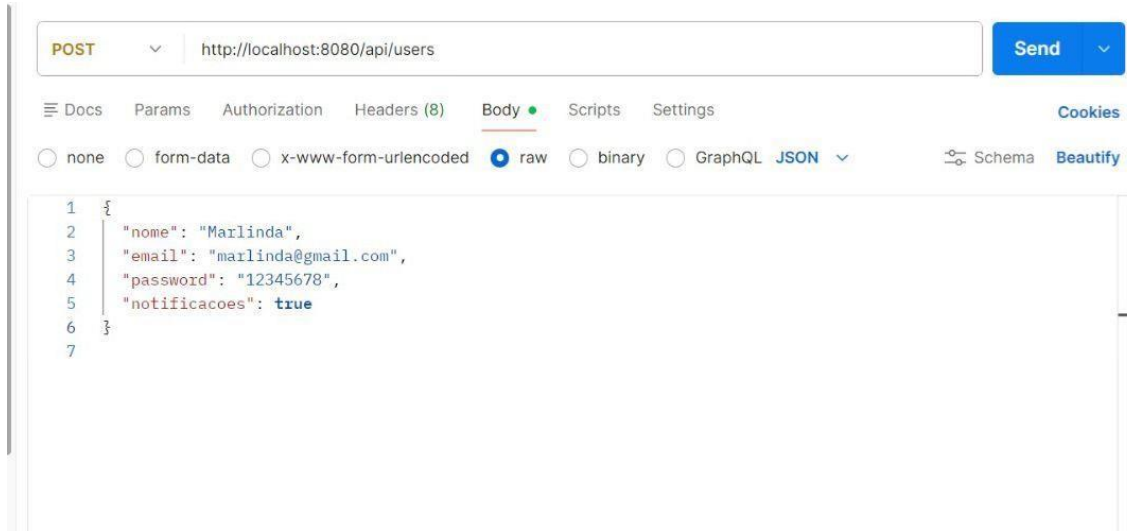
transacao	
id_transacao	BIGINT
id_carteira	INT
id_categoria	BIGINT
valor	DOUBLE
data_transacao	DATE
descricao	VARCHAR(255)
tipo	VARCHAR(255)
localizacao	VARCHAR(255)
latitude	DOUBLE
longitude	DOUBLE
Indexes	

Carteira

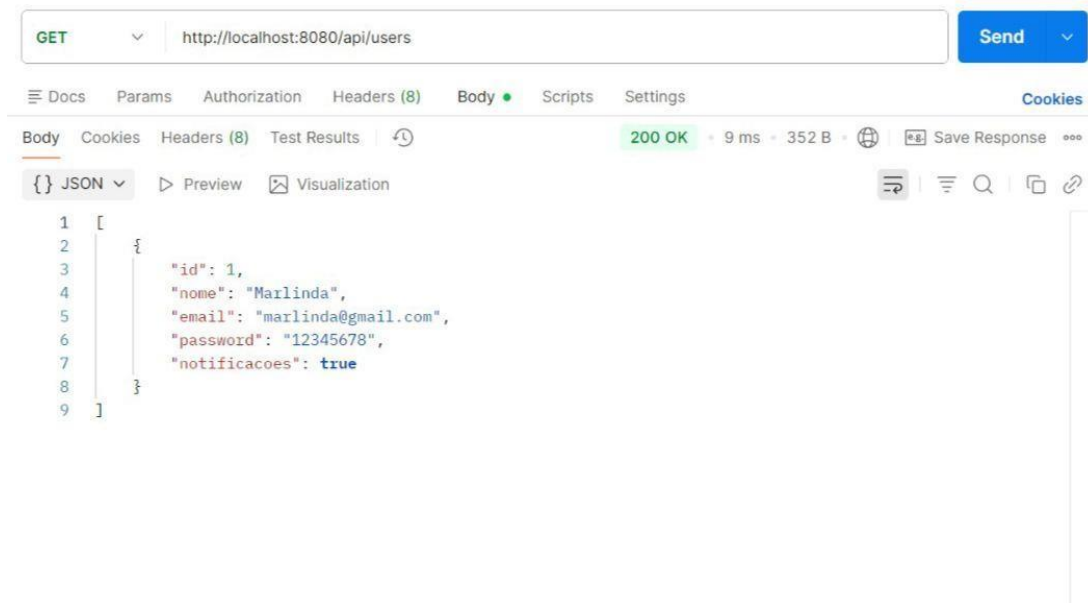
carteira	
id_carteira	INT
id_user	INT
saldo	DOUBLE
Indexes	

Documentação Rest

Criar User



Listar User



Buscar User

GET http://localhost:8080/api/users/1 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

1

Body Cookies Headers (8) Test Results 200 OK 8 ms 350 B Save Response

{ } JSON Preview Visualization

```
1 {
2   "id": 1,
3   "nome": "Marlinda",
4   "email": "marlinda@gmail.com",
5   "password": "12345678",
6   "notificacoes": true
7 }
```

Atualizar User

PUT http://localhost:8080/api/users/1 Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "nome": "Marlinda Silva",
3   "email": "marlinda@gmail.com",
4   "password": "novaSenha123",
5   "notificacoes": true
6 }
7
```

Body Cookies Headers (8) Test Results 200 OK 18 ms 360 B Save Response

{ } JSON Preview Visualization

```
1 {
2   "id": 1,
3   "nome": "Marlinda Silva",
4   "email": "marlinda@gmail.com",
5   "password": "novaSenha123",
6   "notificacoes": true
7 }
```

Deletar User

DELETE ▼ http://localhost:8080/api/users/1 Send ▼

Docs Params Authorization Headers (6) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1
```

Criar Transações

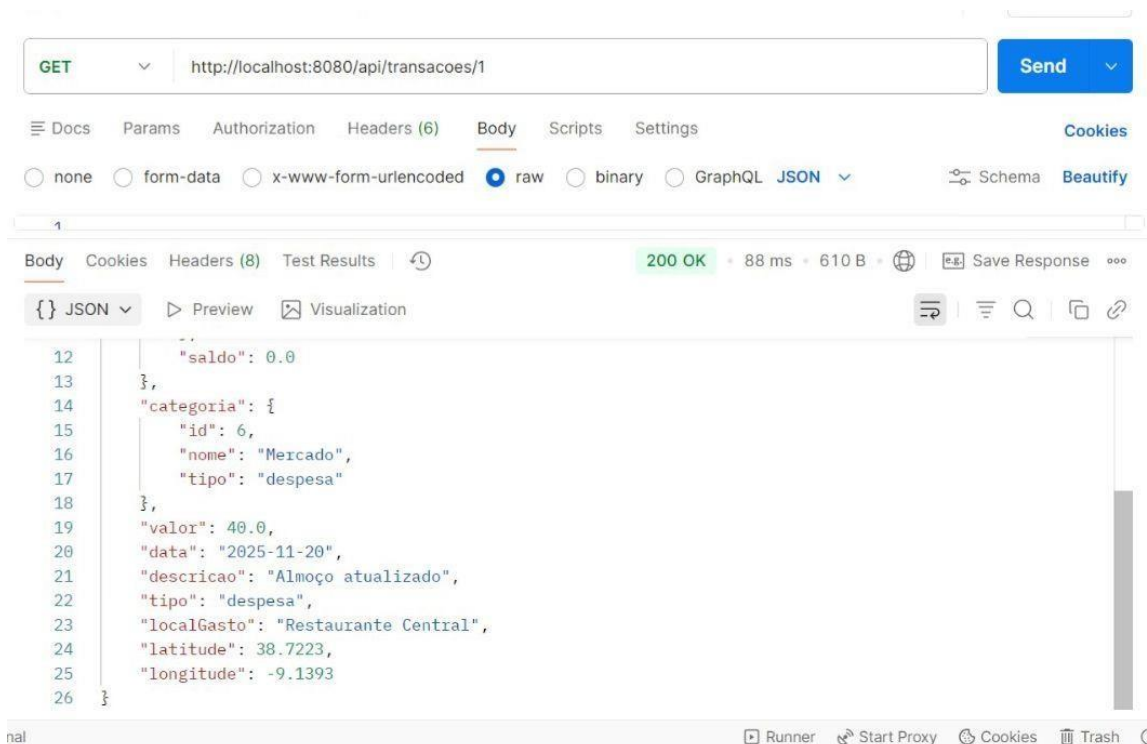
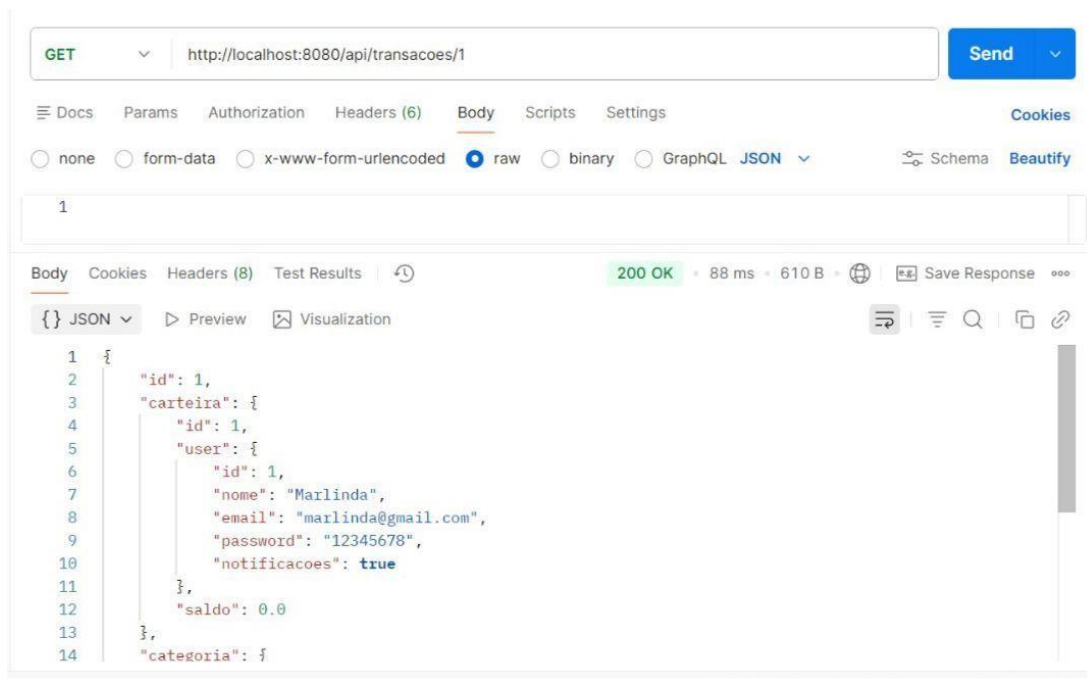
POST ▼ http://localhost:8080/api/transacoes Send ▼

Docs Params Authorization Headers (8) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 {
2   "carteira": {
3     "id": 1
4   },
5   "categoria": {
6     "id": 6
7   },
8   "valor": 25.50,
9   "data": "2025-11-20",
10  "descricao": "Almoço no restaurante",
11  "tipo": "despesa",
12  "localGasto": "Restaurante Central",
13  "latitude": 38.7223,
14  "longitude": -9.1393
15 }
16
17
```

Listar Transações



Buscar Transações

The image displays two screenshots of a REST client interface, likely Swagger Client, showing the results of a GET request to the endpoint `http://localhost:8080/api/transacoes/1`.

Top Screenshot: The response is a single JSON object representing a transaction. The status is **200 OK**, with a response time of 88 ms and a body size of 610 B. The JSON structure is as follows:

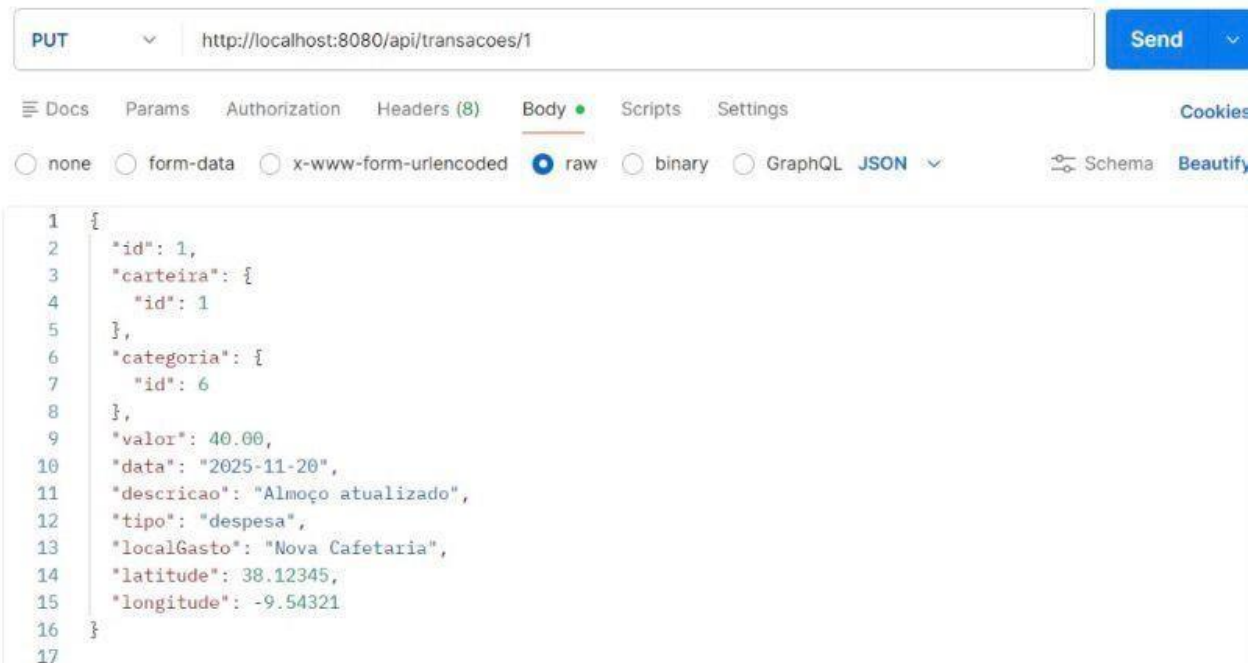
```
1 {
2   "id": 1,
3   "carteira": {
4     "id": 1,
5     "user": {
6       "id": 1,
7       "nome": "Marlinda",
8       "email": "marlinda@gmail.com",
9       "password": "12345678",
10      "notificacoes": true
11    },
12    "saldo": 0.0
13  },
14  "categoria": {
```

Bottom Screenshot: The response is a list of JSON objects. The status is **200 OK**, with a response time of 88 ms and a body size of 610 B. The JSON structure is as follows:

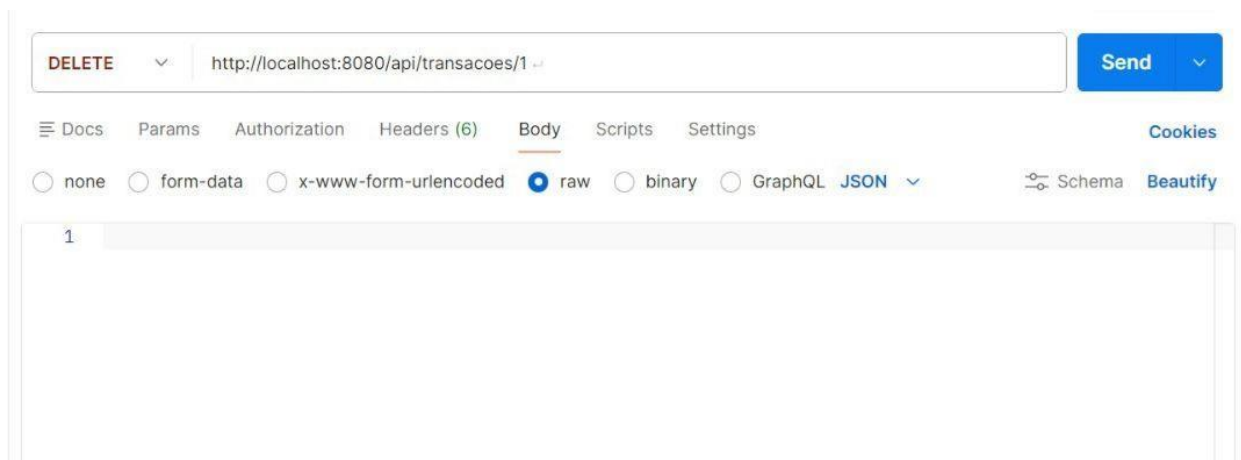
```
12    "saldo": 0.0
13  },
14  "categoria": {
15    "id": 6,
16    "nome": "Mercado",
17    "tipo": "despesa"
18  },
19  "valor": 40.0,
20  "data": "2025-11-20",
21  "descricao": "Almoço atualizado",
22  "tipo": "despesa",
23  "localGasto": "Restaurante Central",
24  "latitude": 38.7223,
25  "longitude": -9.1393
26 }
```

At the bottom of the interface, there are navigation links: **Runner**, **Start Proxy**, **Cookies**, **Trash**, and a help icon.

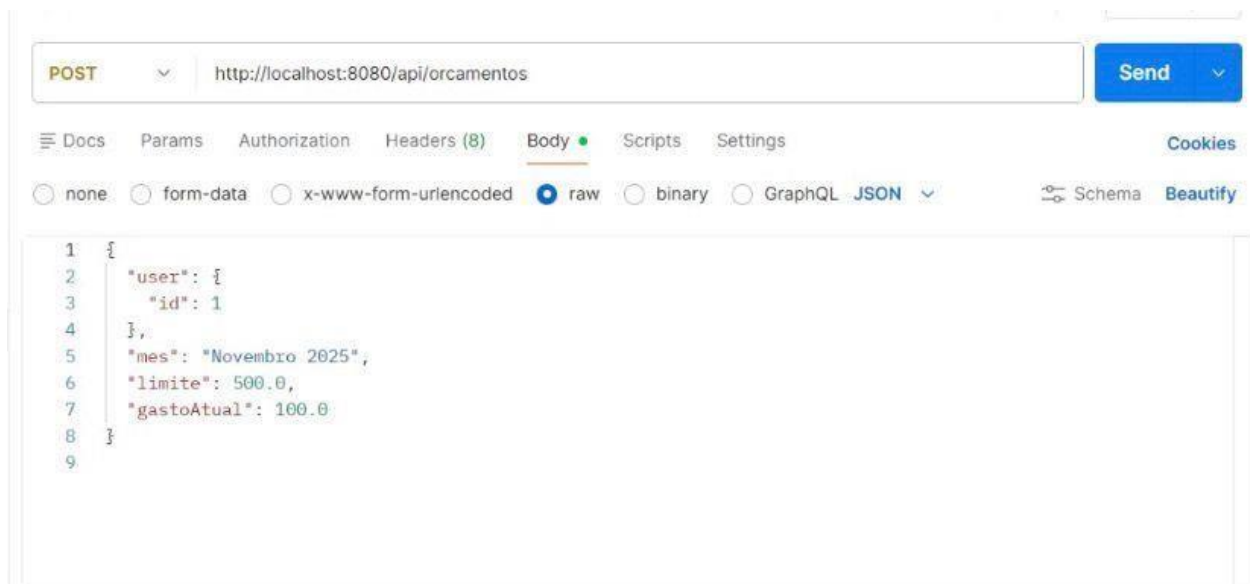
Atualizar Transações



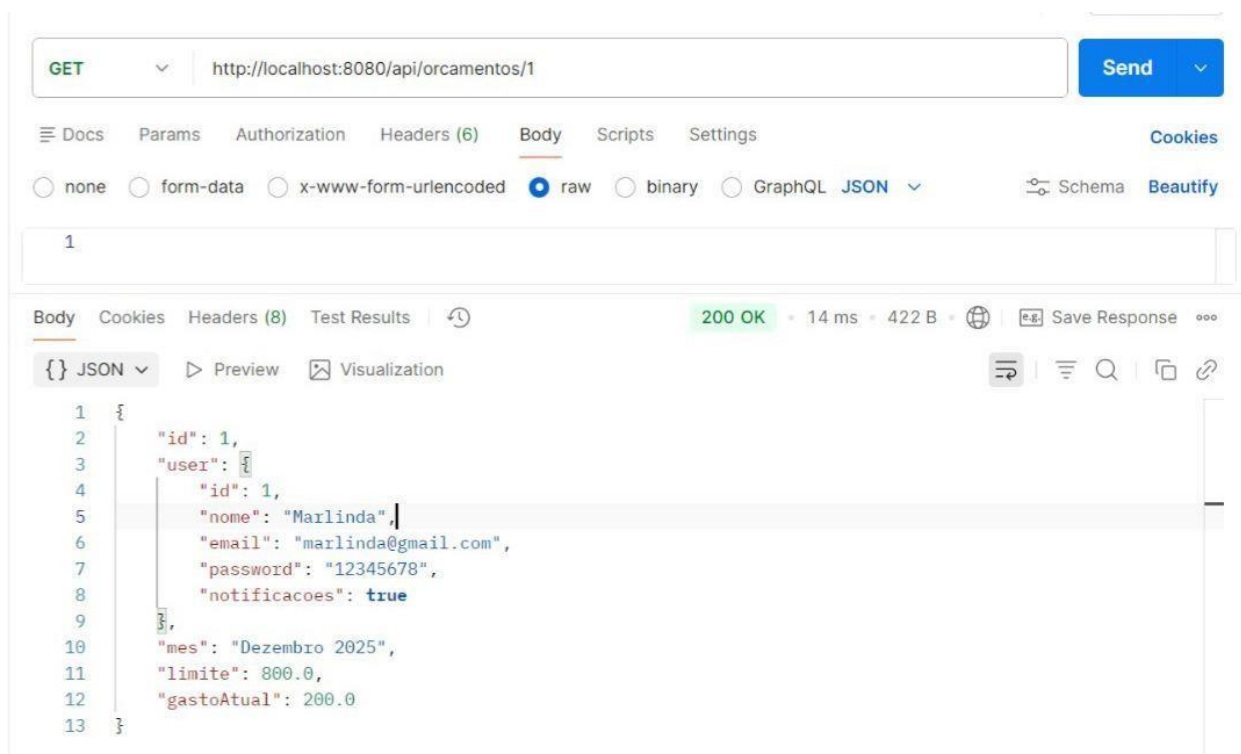
Deletar Transação



Criar Orçamento



Listar Orçamento



Atualizar Orçamento

PUT

http://localhost:8080/api/orcamentos/1

Send

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

Cookies

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL

JSON

Schema

Beautify

```
1 {  
2   "id": 1,  
3   "user": { "id": 1 },  
4   "mes": "Dezembro 2025",  
5   "limite": 800.0,  
6   "gastoAtual": 200.0  
7 }  
8
```

Deletar Orçamento

DELETE ▼ http://localhost:8080/api/orcamentos/1 Send ▼

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Schema Beautify

```
1
```

Criar Meta

POST ▼ http://localhost:8080/api/metadados Send ▼

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Schema Beautify

```
1 {
2   "nome": "Viagem para Paris",
3   "valorObjetivo": 500,
4   "valorAtual": 0,
5   "dataLimite": "2025-12-31",
6   "descricao": "Juntar para a próxima viagem",
7   "user": {
8     "id": 1
9   }
10 }
11
12
```

Listar Meta

GET ▼ http://localhost:8080/api/metastSend ▼

Docs Params Authorization Headers (8) **Body** • Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1  [  
2    {  
3      "id": 1,  
4      "nome": "Viagem para Paris",  
5      "valorObjetivo": 500,  
6      "valorAtual": 0,  
7      "user": {  
8        "id": 1,  
9        "nome": "Marlinda Silva",  
10       "email": "marlinda@gmail.com",  
11       "password": "novaSenha123",  
12       "notificacoes": true,  
13       "dataInicio": null,  
14       "dataFim": "2026-01-10"  
15     }  
16  ]
```

Buscar Meta

GET ▼ http://localhost:8080/api/metast/1Send ▼

Docs Params Authorization Headers (6) **Body** • Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

1

Body Cookies Headers (8) Test Results ⌚ **200 OK** · 29 ms · 496 B · 🌐 💾 Save Response ⋮

{} JSON ▼ ▶ Preview 🖼 Visualization 🔍 📄 🔗

```
1  {  
2    "id": 1,  
3    "user": {  
4      "id": 1,  
5      "nome": "Marlinda Silva",  
6      "email": "marlinda@gmail.com",  
7      "password": "novaSenha123",  
8      "notificacoes": true,  
9    },  
10   "nome": "Viagem atualizada para Paris",  
11   "valorObjetivo": 700.0,  
12   "valorAtual": 100.0,  
13   "dataInicio": null,  
14   "dataFim": "2026-01-10"  
15 }
```

Atualizar Meta

PUT ▼ http://localhost:8080/api/metad/1 Send ▼

Docs Params Authorization Headers (8) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1 {
2   "id": 1,
3   "nome": "Viagem atualizada para Paris",
4   "valorObjetivo": 700,
5   "valorAtual": 100,
6   "dataInicio": null,
7   "dataFim": "2026-01-10",
8   "user": {
9     "id": 1
10  }
11 }
12 }
```

Deletar Meta

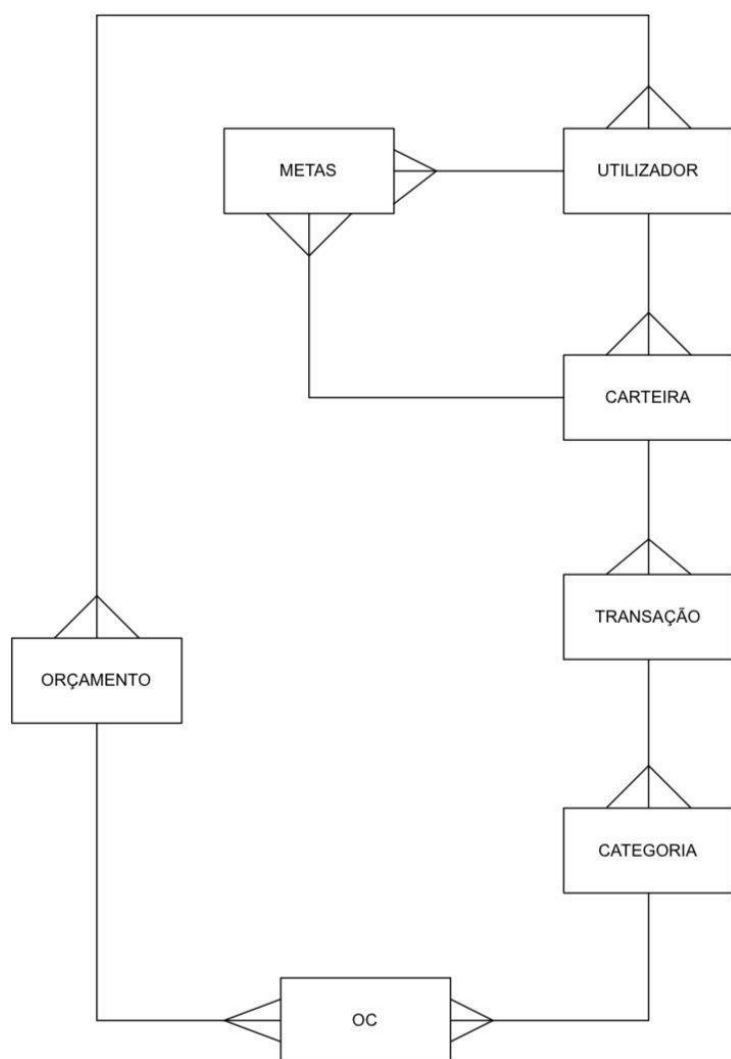
DELETE ▼ http://localhost:8080/api/metad/1 Send ▼

Docs Params Authorization Headers (6) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Schema Beautify

```
1
```

MER



Conclusão

O aplicativo studycash é uma solução prática e útil que surge para ajudar os estudantes a organizar e gerir melhor as suas finanças pessoais. Com esta aplicação móvel simples, moderna e acessível, pretende-se facilitar o registo de receitas e despesas, a criação de orçamentos e metas de poupança, bem como a visualização de relatórios financeiros de forma clara e objetiva.

Com estas funcionalidades, o studycash permitirá o utilizador acompanhar a sua vida financeira em tempo real, tomar decisões mais conscientes e adotar hábitos de gestão responsáveis e sustentáveis.

O seu objetivo principal é tornar a gestão financeira mais simples, acessível e eficaz para os estudantes, promovendo maior controle e equilíbrio económico no dia a dia académico e pessoal.

Bibliografia

Krug, S. (2014). Don't Make Me Think: A Common Sense Approach to Web Usability (3ª ed.). New Riders.

Investopedia. (2023). Personal Finance Basics: A Beginner's Guide to Financial Literacy. Disponível em: <https://www.investopedia.com>

Android Developers. (2024). Build modern Android apps with Jetpack Compose. Disponível em: <https://developer.android.com/jetpack/compose>

Deitel, P. J., & Deitel, H. M. (2020). Android for Programmers: An App-Driven Approach. Pearson Education.

Anexos

Postman: <https://marlindaprata-7003307.postman.co/workspace/Marlinda-Prata's-Workspace~b4a3ad5b-57f3-44ca-8731-0a60058d5a9b/collection/50166243-55db8156-4502-4cd7-8e2f-7c598a129822?action=share&source=copy-link&creator=50166243>

Manual do utilizador:
https://www.canva.com/design/DAG7gYj5vV8/EPgchRq1oYFVpBybrTqyrA/edit?utm_content=DAG7gYj5vV8&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Contribuição Dos Membros da equipe no Projeto

Marlinda Congo

- Contribuiu para o planeamento, pesquisa e desenvolvimento do tema.
- Contribuiu para a criação do Poster, Mockups no Figma e Logotipo.
- Responsável por elaborar todos os relatórios, desde a primeira entrega até a terceira entrega.
- Responsável pela Modelagem da base de dados final (derivação do DER e modelo relacional).
- Responsável pela implementação do backend final (endpoints REST, serviços e entidades).
- Responsável pela criação da base de dados final (criando os ficheiros create, populate e queries).
- Participou no desenvolvimento do frontend.
- Responsável pela conexão do backend com a base de dados.
- Responsável pela criação das personas.
- Responsável por escrever a documentação REST.
- Responsável por fazer a planificação no clickup.

Samuel Ferreira

- Contribuiu para o planeamento, pesquisa e desenvolvimento do tema.
- Contribuiu para a criação do Poster, Mockups no Figma e Logotipo.
- Desenvolveu o frontend em Android Studio.
- Implementou as interfaces de utilizador (UI/UX) e navegação das telas.
- Construiu a comunicação com o backend via Retrofit.
- Criou o dashboard e os componentes visuais da aplicação.
- Auxiliou na criação dos casos de uso e testes finais.
- Contribuiu para correção de bugs e otimização de performance.
- Contribui para a criação da primeira versão base de dados.
- Responsável por atualizar o repositório e importar todos os ficheiros de código fonte no Github.
- Responsável pela criação dos slides da primeira entrega.
- Contribui para a gravação do vídeo.

Constantino Chipopa

- Para primeira entrega tentou contribuir fazendo o gráfico de gantt, porém não consegui por ter encontrado dificuldades.
- Contribui para a criação da primeira versão do Backend e conexão com a base de dados.
- Contribui para a criação da primeira versão do Diagrama ER.
- Responsável pela criação do vídeo.

OBS: O Constantino, durante o desenvolvimento do projeto mostrou-se sempre disposto a ajudar e dar a sua contribuição atempadamente para um bom resultado do projeto em equipe.

Lueji Covilhã

- Para a primeira entrega tentou contribuir com a criação dos slides para a primeira apresentação. Entretanto, os slides feitos por ele não estavam coerentes com o que se pretendia apresentar.
- Contribuiu para a criação da primeira versão do Backend e conexão com a base de dados.
- Auxiliou o Samuel, na conexão do frontend com o backend

OBS: O Lueji, para a primeira e segunda entrega mostrou- se pouco proativo com o desenvolvimento do projeto em equipe. Entretanto, para a terceira entrega mostrou- se mais proativo com o trabalho em equipe.

Gilma Mulanda

- Tentou contribuir no desenvolvimento do frontend, porém teve complicações no seu computador com o Android Studio.
- Contribui com a organização da segunda apresentação oral.
- Contribui para a gravação do vídeo.

OBS: A Gilma, integrou-se no grupo após a primeira entrega. Durante o desenvolvimento do projeto mostrou-se pouco proativa com os trabalhos em equipe. Entretanto, para os ultimos dias tem se mostrado mais presente.

